Date	Version	Description
05/09/2026	1.0	• Création de la notice

I. Contexte

Entity Framework (raccourci en EF) est l'ORM le plus populaire pour le framework .net et c'est celui qu'on va utiliser pour interagir avec la base de donnée dans ce projet

II. Installation

Il faudra installer les nuGets suivants :

- Microsoft.EntityFrameworkCore 10.0.11 **> L'ORM**
- Microsoft.EntityFrameworkCore.Design 10.0.11 **> BDD "CodeFirst"**
- En fonction de la base de données retenues : **> Le connecteur SGBD**
    - Microsoft.EntityFrameworkCore.SqlServer 10.0.11
    - Npgsql.EntityFrameworkCore.PostgreSQL 10.0.3

III. Configuration

EF fournit une classe DBContext qu'il est possible d'hériter et de configurer. C'est cette classe qui permettra de créer et d'interagir avec la base de données.

Pour configurer entity framework il faut :

- Lui indiquer le modèle de donnée utiliser (les tables et leurs relations)
- Lui donner le lien vers la base de données (la connection string)
- L'enregistrer dans l'injecteur de dépendance .net pour qu'il puisse être utilisé partout dans l'application

💡 Remarque : Pour s'y retrouver dans une base de code grossissantes il est conseillé de ranger tous les fichiers en rapport avec la base de données dans un même dossier ("Data", "Infrastructure", "AccessLayer", ... ou autre selon votre préférence d'architecture)

A. Le modèle de données

Le modèle de données se configure dans une classe héritant de DBContext.

```
using FTELMarket_CLI.Infrastructure.Entities;
using Microsoft.EntityFrameworkCore;

namespace FTELMarket_CLI.Infrastructure;

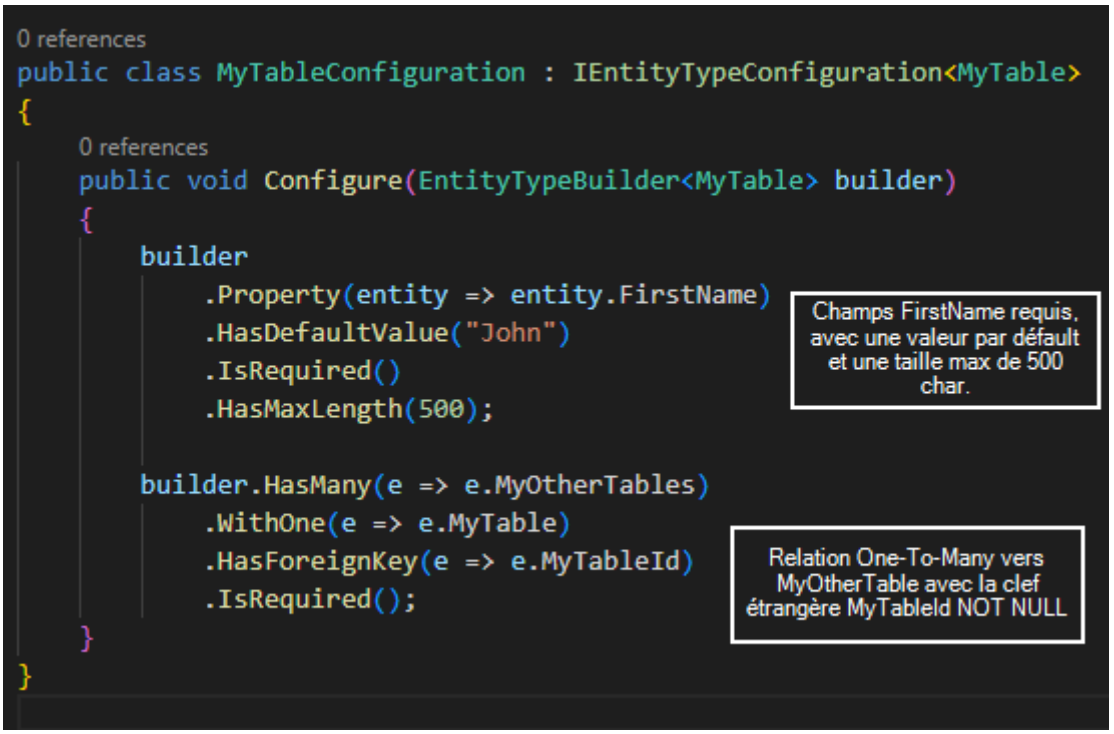
2 references
public class FtelMarketDbContext : DbContext
{
        // Déclaration des tables de la BDD
        0 references
        public DbSet<MyTable> MyTable { get; set; }
        0 references
        public DbSet<MyOtherTable> MyOtherTable { get; set; }

        0 references
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
                modelBuilder.ApplyConfigurationsFromAssembly(typeof(FtelMarketDbContext).Assembly);
        }
}
```

La description/contraintes des tables ainsi que leurs relations peut se configurer de plusieurs manières mais nous utiliserons ici le mode "fluent API" (Référence Microsoft)

Pour cela chaque table aura (si besoin) une classe héritant de IEntityTypeConfiguration<NomDeLaClasse>.

Cet héritage impose l'implémentation d'une méthode "Configure" permettant de manipuler un "entity builder" servant à décrire les contraintes et relation de la table :



B. Le lien vers la base de données

Pour faire le lien avec une vraie base de données il faut que le lien vers cette base puisse être communiqué à EntityFramework. La chaîne de connexion comme beaucoup de paramètre applicatif doit être déclarée dans le fichier appsettings.json

Les conventions .NET prévoient que la clef ConnectionStrings indiquent un ensemble de chaînes de connexions sous le format {"nom de la chaîne": "chaîne de connexion"}.

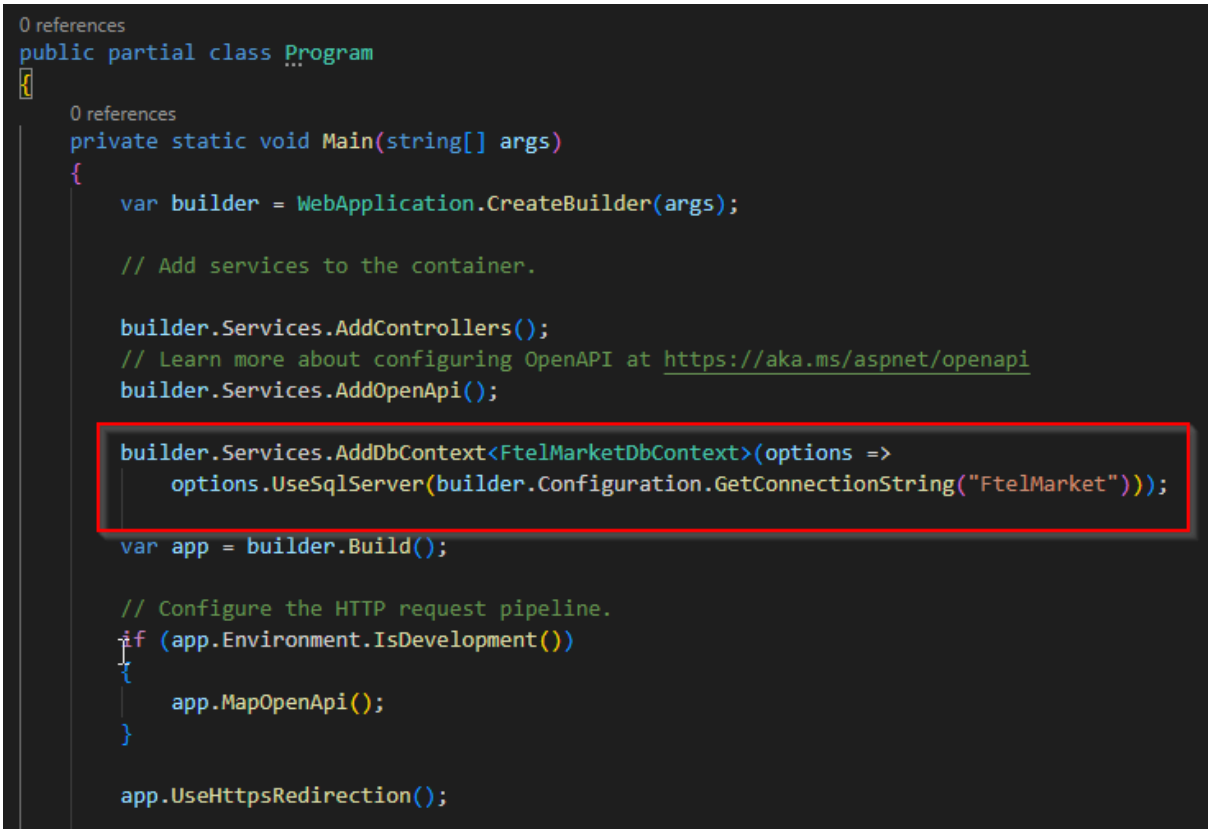


Exemple d'un appsettings.json avec 2 chaines de connexion.

💡 Remarque : Toutes les chaînes de connexion ont un format différent selon les SGBD, le site connectionstrings.com en recence un bon nombre.

C. Enregistrement dans le moteur d'injection de dépendance

Pour que Entity Framework soit utilisable par notre application il faut enregistrer ce nouveau service dans program.cs.



```
0 references
public partial class Program
{
    0 references
    private static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        // Add services to the container.

        builder.Services.AddControllers();
        // Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
        builder.Services.AddOpenApi();

        builder.Services.AddDbContext<FtelMarketDbContext>(options =>
            options.UseSqlServer(builder.Configuration.GetConnectionString("FtelMarket")));

        var app = builder.Build();

        // Configure the HTTP request pipeline.
        if (app.Environment.IsDevelopment())
        {
            app.MapOpenApi();
        }

        app.UseHttpsRedirection();
```

Au lancement du projet FtelMarketDbContext sera appelé, le lien vers la base de données sera testé via la chaîne de connexion “FtelMarket“ et si le serveur de BDD répond le projet se lancera sans erreur.

IV. Exemple de rangement des fichiers

